

Reserve pathways editorial

1. Problem description; story, and relation to MST algorithms

The presented problem hints at the use of an algorithm which produces a *minimum spanning tree*, either *Prim's* algorithm, or *Kruskal's*, in combination with a greedy approach to pick the maximum number of paths possible from the unused paths, in order of cheapest edge weight.

To produce the desired solution, the very network (as described above) is to be yielded, with the use of either of the above algorithms.

Of the paths which are not included in the MST (unused by Kruskal's/Prim's), we are asked to select the maximum number of paths possible which can be implemented, under a constraint T (where the sum of the selected paths is at most T).

2. Intended solution; algorithmic correctness

Kruskal's algorithm is the intended solution to this problem, however, since Prim's algorithm also yields an MST, Prim's is also suitable for this problem.

Kruskal's algorithm always yields an MST, attributable to two fundamental properties:

- The *cycle property*^[1]: Kruskal's algorithm only accepts a path if its endpoints reside within distinct connected components. Hence, when Kruskal's algorithm rejects a path which introduces a cycle, the already-accepted path is less than or equal to the cost of the to-be introduced path, therefore, rejecting the path never makes the MST worse.
- The *cut property*^[2]: the cheapest edge crossing a cut must itself belong to an MST; selecting the cheapest edge connecting two components shall not inhibit us from reaching an optimal solution.

Of note, Prim's algorithm only makes use of the cut property.

Of the edges unused by either algorithm, a greedy approach here is appropriate in ensuring that we can select as many of these edges as possible, given a constraint T (where the sum of the selected edges must be at most T). For the selection of the cheapest path, this shall leave us with the *highest budget* possible after selecting the path; selecting any other path shall leave us with strictly the same budget, *or less*. This means that we can select from a larger set of paths; therefore, always selecting the cheapest possible path shall yield the maximal number of paths which can be implemented into the network.

Prim's and Kruskal's algorithm may produce different MSTs, however, for each MST, the sum of weights is identical, as a result of the fact that each MST has the same multiset of costs^[3], which also means that two distinct MSTs shall leave two sets of unused edges, each with the same multiset of costs, too. Therefore, under any of the algorithms, a greedy approach shall pick the same number of paths to yield the maximum number of selectable paths.

3. Analysis of complexity

The overall time complexity of the algorithm is $O(E \log E)$, since the edge list is sorted by weight; by use of *tim-sort*, *merge-sort*, or any other such optimal comparison-based sorting algorithm, the time complexity holds. However, under separate sorting disciplines, the overall time complexity will vary.

The disjoint-union data structure itself is very fast. The desirable implementation of a disjoint-union set utilizes *path-compression*, and *union-by-rank*, exponentially speeding up the performance of *find*, and *union* operations. Union-by-rank works by attaching the root of the smaller-ranked tree to the root of the larger-ranked tree, hence, reaching a rank k requires at least 2^k nodes, therefore the height of any tree in this data structure is bounded above by $O(\log n)$.

Since any tree within a disjoint-union data structure is bounded by $O(\log n)$ as per union-by-rank, then this means that an initial call to *find*(u) takes $O(\log n)$ time. However, during that initial call, the trees are flattened along the journey to the root of the supplied vertex. This essentially means that subsequent calls shall take $O(1)$ time. Since the trees are constantly flattened, and unioned-by-rank, the amortized time complexity evaluates to $O(\alpha(n))$ (where $\alpha(n)$ is the *inverse Ackermann function* on an input of size n), which is an extremely slowly growing function.

The edge-list stores the set of all edges; hence the space complexity is $O(E)$. Coupled with the fact that the disjoint-union data structure requires $O(V)$ space for the rank, and $O(V)$ for the parent list, the overall space complexity of the disjoint-union data structure is $O(V) + O(V) = O(2V) \sim O(V)$, therefore, the total space complexity is $O(V + E)$ overall for the algorithm.

4. AI use declaration

AI was used in moderation to provide guidance on this implementation, that includes:

- Tools and setup; specifically, assistance with installing problemtools via WSL w/ pipx, installing PyPy, and command-line usage.
- Clarification in debugging “*ERROR: INPUT VALIDATOR NOT FOUND*”.
- Exploring naïve cases which yielded pseudo-correct outputs for sample data, but wrong outputs for secret data.
- Offering conceptual guidance on debugging.
- Explaining the implications behind a recursive discipline on time-out-based submissions.
- Review; reviewed the .tex file specifically for formatting issues.

AI was used entirely for self-reasoning purposes; hence the submitted work is my own doing.

5. References

- [1] M. I. Hadjoudj, “Kruskal’s MST Algorithm Explained”, *Algorithm Visualiser*. [Online]. Available: <https://learngraphtheory.org/articles/kruskals-algorithm.html> [Accessed: Oct. 1, 2026].
- [2] the computer science teacher. “Understanding Algorithms (Optimization Structures), Part 23: Kruskal’s Algorithm”, *Medium*, Feb. 16, 2026 [Online]. Available: <https://parashar--manas.medium.com/understanding-algorithms-optimization-structures-part-23-kruskals-algorithm-f250ae18f4ac> [Accessed: Oct. 1, 2026].
- [3] “Minimum Spanning Tree”, *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Minimum_spanning_tree#Possible_multiplicity [Accessed: Oct. 1, 2026].